

memoize <pre>function memoize(fn) { const cache = {}; return function (x) { if (cache[x]) return cache[x]; return cache[x] = fn(x); }; }</pre>	flatten <pre>function flatten(arr) { return arr.reduce((acc, val) => Array.isArray(val) ? acc.concat(flatten(val)) : acc.concat(val), []); }</pre>	twoSum <pre>function twoSum(nums, target) { const map = new Map(); for(let i=0 ; i<nums.length ; i++) { const complement = target - nums[i]; if(map.has(complement)) { return [map.get(complement), i]; } map.set(nums[i], i) } }</pre>
Clouser <pre>function outer() { let count = 0; return function inner() { count++; console.log(count); }; } const counter = outer(); counter(); // 1 counter(); // 2</pre>	throttle <pre>function throttle(fn, limit) { let flag = true; return function (...args) { if (flag) { fn.apply(this, args); flag = false; setTimeout(() => { flag = true; }, limit); } }; }</pre>	removeDuplicates <pre>function removeDuplicates(arr) { let result = []; for (let i = 0; i < arr.length; i++) { if (!result.includes(arr[i])) { result.push(arr[i]); } } return result; }</pre>
groupAnagrams <pre>function groupAnagrams(strs) { const map = new Map(); for (const s of strs) { const key = s.split('').sort().join(""); if (!map.has(key)) map.set(key, []); map.get(key).push(s); } return [...map.values()]; }</pre>	lengthOfLongest <pre>const s = "abcabcb"; function lengthOfLongestSubstringBruteForce(s) { let max = 0; for (let i = 0; i < s.length; i++) { const set = new Set(); for (let j = i; j < s.length; j++) { if (set.has(s[j])) { break; } set.add(s[j]); max = Math.max(max, j - i + 1); } } return max; }</pre> console.log(lengthOfLongestSubstringBruteForce(s));	debounce <pre>function debounce(fn, delay) { let timerId; return function (...args) { clearTimeout(timerId); timerId = setTimeout(() => { fn.apply(this, args); }, delay); }; } function search(query) { console.log(`Searching for \${query}`); } const debouncedSearch = debounce(search, 500); debouncedSearch('j'); debouncedSearch('js'); debouncedSearch('javascript');</pre>
Valid-parenthesis <pre>function isValid(s) { const stack = []; const map = { ')': '(', ']': '[', '}' : '{' }; for (const c of s) { if ('({['.includes(c)) { stack.push(c); } else { if (stack.pop() !== map[c]) return false; } } return stack.length === 0; } // Time: O(n) Space: O(n) const s = "()[]{}"; console.log(isValid(s)); // Output: true</pre>	maxSubArray <pre>function maxSubArray(nums) { let curr = nums[0], max = nums[0]; for (let i = 1; i < nums.length; i++) { curr = Math.max(nums[i], curr + nums[i]); max = Math.max(max, curr); } return max; }</pre>	binarySearch <pre>function binarySearch(nums, target) { let left = 0; let right = nums.length - 1; while (left <= right) { const mid = Math.floor((left + right) / 2); if (nums[mid] === target) return mid; if (nums[mid] < target) left = mid + 1; else right = mid - 1; } return -1; } console.log(binarySearch([-1, 0, 3, 5, 9, 12], 9)); // 4 console.log(binarySearch([-1, 0, 3, 5, 9, 12], 2)); // -1</pre>